

República Bolivariana de Venezuela
Ministerio del poder popular para la educación superior
Universidad politécnica territorial José Antonio Anzoátegui
Guanta estado Anzoátegui



Base de Datos

Profesor: Humner Lenis

Bachiller: José Suniaga

CI: 28.702.039

8 de Junio de 2026

Las bases de datos constituyen uno de los pilares fundamentales de la informática moderna y representan el núcleo central donde se almacena, gestiona y procesa la información en prácticamente todas las aplicaciones empresariales, científicas y personales que utilizamos en la actualidad. Desde los sistemas bancarios que manejan nuestras transacciones financieras hasta las redes sociales que conectan a millones de personas en todo el mundo, todos estos sistemas dependen de mecanismos robustos y eficientes de almacenamiento de datos.

Este informe tiene como objetivo proporcionar una cobertura exhaustiva de los conceptos fundamentales relacionados con las bases de datos, abarcando desde los sistemas manejadores de bases de datos hasta el lenguaje SQL utilizado para interactuar con ellas. El contenido se estructura en cuatro unidades principales que abordan progresivamente los diferentes aspectos de esta disciplina, desde los conceptos más generales y teóricos hasta las aplicaciones prácticas de consulta y manipulación de datos.

A lo largo de este documento, el lector encontrará una explicación detallada de los componentes y arquitectura de los sistemas de bases de datos, los modelos conceptuales utilizados para representar la información, el modelo relacional que constituye la base de la mayoría de los sistemas modernos, y finalmente el lenguaje SQL que permite interactuar directamente con los datos almacenados. Sin más preámbulos, procedemos a desarrollar cada una de estas áreas de conocimiento.

Concepto de Sistema de Base de Datos y Sistema Manejador de Base de Datos (SMBD)

Un sistema de base de datos puede definirse como una colección organizada e integrada de datos relacionados entre sí, que se almacena de manera persistente en un dispositivo de almacenamiento secundario, y que es gestionada por un conjunto de software especializado capaz de manejar múltiples operaciones de acceso, modificación y consulta. La base de datos propiamente dicha representa la abstracción que permite coordinar la forma en que los datos se almacenan físicamente de la forma en que los usuarios y aplicaciones los perciben y manipulan.

La característica fundamental que diferencia a una base de datos de un simple conjunto de archivos es la existencia de un sistema manejador de base de datos, conocido internacionalmente por sus siglas en inglés como DBMS (Database Management System). Este software intermedio, al que denominaremos SMBD, actúa como interfaz entre los usuarios y los datos almacenados, proporcionando una capa de abstracción que oculta la complejidad del almacenamiento físico y

que ofrece un conjunto de herramientas y lenguajes para definir, manipular y consultar la información de manera eficiente y segura.

Las funciones esenciales que cumple un SDBD incluyen la definición de estructuras de datos mediante el lenguaje de definición de datos (DDL), la manipulación de datos a través del lenguaje de manipulación de datos (DML), el mantenimiento de la integridad y consistencia de los datos, la gestión de accesos concurrentes por múltiples usuarios, los mecanismos de recuperación ante fallos, y la optimización del rendimiento de las consultas. En definitiva, el SDBD constituye el corazón de cualquier sistema de bases de datos y determina en gran medida las capacidades, rendimiento y confiabilidad del sistema completo.

La evolución histórica de los sistemas de bases de datos ha sido notable y ha pasado por varias etapas claramente diferenciadas. Los primeros sistemas de bases de datos, desarrollados en las décadas de 1960 y 1970, eran fundamentalmente sistemas de archivos mejorados que utilizaban modelos jerárquicos o en red, siendo el IMS de IBM uno de los ejemplos más representativos de esta era. Estos sistemas, aunque revolucionarios para su época, presentaban limitaciones significativas en términos de flexibilidad y dependencia de la estructura física de los datos.

La verdadera revolución en el campo llegó con los trabajos pioneros de Edgar F. Codd, quien en 1970 publicó su artículo fundacional "A Relational Model of Data for Large Shared Data Banks", en el que propuso un modelo teórico basado en la teoría de conjuntos y el álgebra relacional que separaba completamente la estructura lógica de los datos de su almacenamiento físico. Este modelo relacional se convirtió rápidamente en el paradigma dominante y dio lugar al desarrollo de sistemas manejadores de bases de datos relacionales (RDBMS) como Oracle, DB2, Ingres y posteriormente MySQL, PostgreSQL y SQL Server.

Evolución de los Sistemas Manejadores de Base de Datos

La historia de los SDBD puede dividirse en varias generaciones claramente diferenciadas, cada una de las cuales introdujo mejoras sustanciales en términos de capacidades, rendimiento y facilidad de uso. La primera generación, correspondiente a los sistemas jerárquicos y de red de los años 1960 y 1970, desarrolló los conceptos fundamentales de independencia de datos y acceso estructurado, aunque estos sistemas requerían que los programadores conocieran íntimamente la estructura física de almacenamiento.

La segunda generación, propiedad con el modelo relacional propuesto por Codd, revolucionó la industria al proporcionar un modelo teórico robusto basado en la lógica de predicados y el álgebra relacional. El lenguaje SQL (Structured Query

Language), desarrollado originalmente por IBM como parte del proyecto System R, se convirtió en el estándar de la industria para la manipulación de datos relacionales y sigue siendo, más de cuatro décadas después, el lenguaje más utilizado para interactuar con bases de datos. Los sistemas de esta generación, como Oracle, Informix, Sybase y DB2, introdujeron el concepto de independencia lógica de los datos y permitieron que los usuarios formularan consultas de manera declarativa sin necesidad de conocer la estructura física de almacenamiento.

La tercera generación, cual comenzó a principios de los años 1990, introdujo capacidades orientadas a objetos y características de bases de datos objeto-relacionales. Sistemas como Oracle8 y PostgreSQL permitieron definir tipos de datos complejos, herencias y métodos asociados a clases de objetos, ampliando significativamente las capacidades del modelado de datos. Aunque el paradigma orientado a objetos no terminó de desplazar al modelo relacional, muchas de sus características fueron incorporadas a los sistemas existentes en lo que se conoce como modelo objeto-relacional.

La cuarta generación, que atraviesa desde finales de los años 2000 hasta la actualidad, se caracteriza por el surgimiento de los sistemas NoSQL (No solo SQL), que proponen modelos de datos alternativos al relacional para abordar casos de uso específicos como el almacenamiento de grandes volúmenes de datos no estructurados, las aplicaciones web de alta disponibilidad y el procesamiento de datos a escala masiva. Bases de datos documentales como MongoDB, sistemas clave-valor como Redis, bases de datos de gráficos como Neo4j y sistemas de ancho de columna como Cassandra representan esta nueva generación de sistemas que complementan, más que sustituyen, a las bases de datos relacionales tradicionales.

Más recientemente, el concepto de bases de datos multimodelo ha ganado tracción, permitiendo que un mismo sistema soporte múltiples paradigmas de datos (relacional, documento, gráfico, clave-valor) en una única plataforma. Los ejemplos notables incluyen ArangoDB, Cosmos DB de Microsoft y la propia extensión de PostgreSQL con sus tipos JSON. Esta tendencia refleja la creciente complejidad de los requisitos de almacenamiento de datos en las aplicaciones modernas y la necesidad de elegir el modelo más adecuado para cada tipo de información.

Sistemas de Base de Datos Relacionales

Los sistemas de bases de datos relacionales constituyen la implementación práctica del modelo teórico propuesto por Codd y representan, con diferencia, el tipo de SDBD más utilizado en la actualidad. En este modelo, los datos se organizan en relaciones, que son tablas bidimensionales compuestas por filas

(denominadas doble o registros) y columnas (denominadas atributos o campos). Cada tabla representa una entidad del mundo real o una relación entre entidades, y cada fila representa una instancia específica de esa entidad o relación.

Los sistemas relacionales se fundamentan en principios teóricos sólidos derivados del álgebra relacional y la lógica de predicados de primer orden, lo que garantiza que las operaciones sobre los datos sean matemáticamente correctas y consistentes. El lenguaje SQL, aunque ha evolucionado significativamente desde sus orígenes, sigue siendo el estándar de la industria para interactuar con estos sistemas y proporciona capacidades para definir esquemas de datos, manipular información, realizar consultas complejas y establecer controles de seguridad e integridad.

Las ventajas del modelo relacional incluyen su simplicidad conceptual, que facilita la comprensión y el diseño de estructuras de datos; su base teórica sólida, que permite razonar formalmente sobre las operaciones y propiedades de los datos; su independencia de la estructura física, que permite modificar el almacenamiento sin afectar a las aplicaciones; y su lenguaje estandarizado, que proporciona portabilidad entre diferentes sistemas. Estas características han convertido al modelo relacional en la opción predominante para la mayoría de las aplicaciones empresariales y de negocio durante más de cuatro décadas.

Arquitecturas Cliente-Servidor y Arquitecturas Multi-capas

La arquitectura cliente-servidor representa el modelo Fundamental de organización de los sistemas de bases de datos modernos y establece una separación clara entre las funciones de presentación, que se ejecutan en el cliente; las funciones de procesamiento de lógica de negocio y datos, que se ejecutan en el servidor; y las funciones de almacenamiento, que pueden estar integradas en el servidor de bases de datos o separadas en dispositivos de almacenamiento dedicados.

En el modelo cliente-servidor tradicional, el cliente envía peticiones en forma de consultas SQL al servidor de bases de datos, quien las procesa, ejecuta y devuelve los resultados al cliente. Esta separación permite optimizar cada componente según sus necesidades específicas: los clientes pueden ejecutarse en estaciones de trabajo o dispositivos menos potentes concentrados en la interfaz de usuario, mientras que el servidor puede configurarse con recursos importantes de procesamiento, memoria y almacenamiento para manejar cargas de trabajo intensivo.

Las ventajas de esta arquitectura incluyen la centralización del control de datos, que facilita la implementación de políticas de seguridad y consistencia; la

reducción del tráfico de red al enviar únicamente las consultas y resultados, en lugar de transmisión de archivos completos; la facilidad de actualización y mantenimiento al modificar únicamente el servidor; y la escalabilidad gradual mediante la adición de más clientes o la actualización del servidor según las necesidades crecientes.

La evolución de esta arquitectura ha llevado al desarrollo de modelos multicapas que introducen una capa adicional de aplicación o middleware entre el cliente y el servidor de bases de datos. Esta capa intermedia, que puede implementarse mediante servidores de aplicaciones como JBoss, WebSphere o IIS, se encarga de manejar la lógica de negocio, la validación de datos, la gestión de transacciones distribuidas y la conexión a múltiples fuentes de datos. Este modelo es especialmente útil en aplicaciones web empresariales y sistemas distribuidos que requieren alta disponibilidad y escalabilidad.

En las arquitecturas más modernas basadas en servicios web y microservicios, cada componente de la aplicación se expone como un servicio independiente que puede desplegarse y escalarse de forma autónoma. Las bases de datos, en este contexto, se convierten en servicios especializados que proporcionan APIs específicas para acceder a los datos, ya sea mediante interfaces SQL tradicionales o mediante APIs propietarias de cada sistema. Esta tendencia ha impulsado el desarrollo de bases de datos como servicio (DBaaS) y la proliferación de servicios gestionados en la nube como Amazon RDS, Azure SQL Database y Google Cloud SQL.

Componentes de un DBMS

Un sistema manejador de bases de datos está compuesto por múltiples componentes que trabajan de manera coordinada para proporcionar todas las funcionalidades requeridas para el almacenamiento, recuperación y gestión de datos. Cada uno de estos componentes cumple un papel específico y juntos constituyen la arquitectura integral del sistema.

El gestor de archivos es responsable de la interacción con el sistema operativo y los dispositivos de almacenamiento físico, manipulando los archivos donde se almacenan los datos y los índices. Gestiona el espacio de almacenamiento, el posicionamiento de los datos en el disco, el acceso secuencial y aleatorio a los registros, y la coordinación con los dispositivos de almacenamiento secundarios. Este componente es fundamental para el rendimiento global del sistema, ya que constituye la interfaz entre la base de datos lógica y el almacenamiento físico.

El gestor de datos (data manager) también conocido como gestor de la base de datos, es el componente central que coordina todas las operaciones de lectura y

escritura. Implementa los mecanismos de control de concurrencia, asegurando que múltiples usuarios puedan acceder simultáneamente a los datos sin generar inconsistencias. También maneja la recuperación ante fallos, garantizando que la base de datos pueda restaurarse a un estado consistente después de una falla del sistema o una caída de la energía.

El gestor de transacciones supervisa la ejecución de las transacciones, que son unidades lógicas de trabajo que agrupan una o más operaciones que deben ejecutarse de manera atómica (o todo o nada) y con garantías de consistencia. Implementa las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) que caracterizan a las transacciones válidas y coordina el registro de transacciones (log) para permitir la recuperación.

El procesador de consultas es el componente responsable de interpretar y ejecutar las consultas expresadas en SQL u otros lenguajes de alto nivel. Este componente analiza la sintaxis de la consulta, la verifica contra el esquema de la base de datos, la traduce a una representación intermedia, optimiza la consulta seleccionando el plan de ejecución más eficiente, y finalmente ejecuta el plan recuperando o modificando los datos solicitados.

El gestor de catálogo mantiene el diccionario de datos o catálogo del sistema, que contiene los metadatos que describen la estructura de la base de datos: definiciones de tablas, columnas, tipos de datos, restricciones, índices, usuarios y permisos. Este componente es fundamental para el funcionamiento del resto de los módulos, ya que todos ellos consultan el catálogo para obtener información sobre la estructura de los datos.

El gestor de autorizaciones controla el acceso a los datos verificando que los usuarios tengan los permisos necesarios para realizar las operaciones solicitadas. Este módulo implementa los mecanismos de seguridad del sistema, autenticando a los usuarios y verificando sus privilegios sobre los objetos de la base de datos.

Funcionalidades de DBMS

Las funcionalidades proporcionadas por un SDBD se pueden clasificar en varias categorías fundamentales que abarcan todos los aspectos de la gestión de datos. Estas funcionalidades constituyen el valor agregado de utilizar un sistema de bases de datos frente a implementar mecanismos de almacenamiento propietario.

La funcionalidad de definición de datos permite crear, modificar y eliminar las estructuras que contendrán los datos. Mediante el lenguaje de definición de datos (DDL), los usuarios pueden especificar las tablas que compondrán la base de datos, los atributos de cada tabla, los tipos de datos de cada atributo, las restricciones de integridad, los índices y otros objetos de la base de datos. Esta

funcionalidad establece el esquema de la base de datos, que define la estructura lógica de la información.

La funcionalidad de manipulación de datos permite insertar, actualizar, eliminar y recuperar información de la base de datos. Mediante el lenguaje de manipulación de datos (DML), los usuarios pueden realizar operaciones sobre los datos individuales o sobre conjuntos de registros que satisfacen ciertos criterios. Esta funcionalidad constituye la base de la interacción cotidiana con la base de datos.

La funcionalidad de consultas permite formular preguntas complejas sobre los datos sin necesidad de especificar el procedimiento para obtener la respuesta. Los lenguajes de consulta declarativos, como SQL, permiten expresar lo que se desea saber, dejando a SDBD la responsabilidad de determinar la forma más eficiente de ejecutar la consulta. Esta separación entre la especificación del resultado y el método de obtención es una de las características más poderosas de los sistemas relacionales.

La funcionalidad de control de concurrencia permite que múltiples usuarios accedan simultáneamente a la base de datos sin generar inconsistencias. Los SDBD implementan mecanismos de bloqueo y control de versiones que garantizan que las transacciones concurrentes produzcan resultados correctos, evitando problemas como la lectura sucia, las actualizaciones perdidas y las inconsistencias lecturas no repetibles.

La funcionalidad de recuperación garantiza que la base de datos pueda restaurarse a un estado consistente después de una falla del sistema, ya sea por errores de software, caídas de energía o desastres físicos. Los mecanismos de registro de transacciones (registro de escritura anticipada), puntos de verificación (puntos de control) y copias de seguridad permiten recuperar los datos incluso después de fallos catastróficos.

La funcionalidad de seguridad protege los datos contra accesos no autorizados mediante mecanismos de autenticación de usuarios, definición de permisos y cifrado. Estas capacidades son especialmente importantes en entornos empresariales donde los datos pueden contener información sensible.

Comandos del Lenguaje de Definición de Datos (DDL)

El lenguaje de definición de datos (DDL, por sus siglas en inglés Data Definición Language) constituye el subconjunto del SQL utilizado para crear, modificar y eliminar las estructuras que definen la base de datos. Los comandos DDL operan sobre el esquema de la base de datos, modificando el catálogo del sistema y generando efectos persistentes sobre la estructura de los datos almacenados.

El comando CREATE se utiliza para crear nuevos objetos en la base de datos, como tablas, índices, vistas, procedimientos almacenados y otros objetos. Para crear una tabla, el comando especifica el nombre de la tabla y la definición de cada columna, incluyendo el nombre del atributo, el tipo de dato y las restricciones aplicables. Por ejemplo, el comando para crear una tabla de empleados incluiría la especificación de columnas como el identificador del empleado, nombre, apellido, categoría laboral, salario y otros atributos relevantes.

Los tipos de datos disponibles varían según el sistema SDBD utilizado, pero generalmente incluyen tipos numéricos (INTEGER, BIGINT, DECIMAL, FLOAT), tipos de cadenas de caracteres (CHAR, VARCHAR, TEXT), tipos de fecha y hora (DATE, TIME, TIMESTAMP), tipos booleanos y otros tipos especializados. La selección apropiada de tipos de datos es fundamental para garantizar el almacenamiento eficiente y la manipulación correcta de la información.

El comando ALTER permite modificar la estructura de objetos existentes, como agregar o eliminar columnas, cambiar tipos de datos, agregar o eliminar restricciones y renombrar objetos. Esta capacidad de modificar el esquema sin necesidad de reconstruir la base de datos completa es esencial para la evolución de las aplicaciones y el mantenimiento de los sistemas en producción.

El comando DROP elimina objetos de la base de datos, como tablas completas con todos sus datos, índices, vistas y otros objetos. Este comando debe utilizarse con precaución, ya que la eliminación de objetos es generalmente irreversible y puede afectar a otros objetos que dependen del objeto eliminado.

El comando TRUNCATE proporciona una forma eficiente de eliminar todos los registros de una tabla sin eliminar la estructura de la tabla en sí. Esta operación es generalmente más rápida que eliminar filas individualmente mediante DELETE porque evita el registro de transacciones para cada fila eliminada.

Las restricciones son elementos fundamentales del DDL que permiten definir reglas de integridad sobre los datos. Las restricciones más comunes incluyen la clave primaria (PRIMARY KEY), que identifica de manera única cada registro en una tabla y cuyos valores no pueden ser nulos ni repetidos; la restricción única (UNIQUE), que garantiza que todos los valores de una columna o conjunto de columnas sean distintos entre sí; la restricción de no nulo (NOT NULL), que obliga a que una columna contenga siempre un valor; la restricción de verificación (CHECK), que permite definir condiciones personalizadas que deben cumplir los valores; y la clave foránea (FOREIGN KEY), que establece una relación entre dos tablas y garantiza la integridad referencial.

El comando CREATE INDEX permite crear estructuras de índice que aceleran las búsquedas y operaciones de recuperación de datos. Los índices son estructuras de datos adicionales que el SDBD utiliza para localizar rápidamente los registros sin necesidad de escanear toda la tabla. Aunque mejoran significativamente el rendimiento de las consultas, los índices consumen espacio de almacenamiento y deben actualizarse cada vez que se modifican los datos, por lo que su uso debe equilibrarse cuidadosamente.

Procesamiento de Consultas

El procesamiento de consultas es una de las funciones más importantes de un SDBD y determina en gran medida el rendimiento percibido por los usuarios. Cuando un usuario formula una consulta en SQL, el sistema debe realizar una serie de pasos para transformarla en operaciones concretas sobre los datos almacenados.

El procesador de consultas lleva a cabo estas tareas. En primer lugar, realice el análisis léxico y sintáctico de la consulta, verificando que la sintaxis sea correcta e identificando los diferentes elementos de la consulta como palabras clave, operadores, nombres de tablas y columnas. Si la consulta contiene errores sintácticos, el procesador devuelve un mensaje de error informativo que ayuda al usuario a corregir.

A continuación, el procesador realiza la validación semántica, verificando que los objetos referenciados (tablas, columnas) existen realmente en la base de datos y que el usuario tiene permisos para acceder a ellos. Esta fase también incluye la resolución de nombres, determinando a qué tabla pertenece cada columna cuando el nombre es ambiguo.

Después de la validación, la consulta se traduce a una representación intermedia, sustancialmente en forma de árbol de expresiones o estructura similar, que captura la semántica de la consulta de manera independiente del sistema de almacenamiento específico. Esta representación intermedia facilita las transformaciones posteriores que realizan el optimizador de consultas.

El optimizador de consultas es el componente más crítico del procesador de consultas y determina la eficiencia con que se ejecutará la consulta. Dado que generalmente existen múltiples formas de ejecutar una consulta y los mismos resultados, el optimizador debe seleccionar el plan de ejecución más eficiente en términos de tiempo de procesamiento y uso de recursos.

Los optimizadores modernos utilizan técnicas sofisticadas que incluyen el análisis de estadísticas sobre los datos almacenados (como la distribución de valores, la cardinalidad de las tablas y la existencia de índices), la aplicación de reglas de

transformación de consultas (como la apretar de selecciones y proyecciones), y la estimación del costo de diferentes planos de ejecución. El resultado es un plan detallado que especifica el orden de las operaciones, los índices a utilizar y los métodos de unión entre tablas.

El ejecutor de consultas lleva a cabo el plan de ejecución generado por el optimizador, recuperando o modificando los datos según lo solicitado por la consulta. Durante la ejecución, el ejecutor puede interactuar con el gestor de almacenamiento para leer bloques de datos del disco, con el gestor de transacciones para mantener la coherencia, y con el gestor de bloqueos para coordinar el acceso concurrente.

Procesamiento de Transacciones

Una transacción es una unidad lógica de trabajo que agrupa una o más operaciones sobre la base de datos que deben ejecutarse de manera atómica, es decir, todas las operaciones se completan exitosamente o ninguna de ellas se aplica. Las transacciones son fundamentales para mantener la consistencia e integridad de los datos en entornos multiusuario donde múltiples operaciones pueden ejecutarse simultáneamente.

Las propiedades fundamentales de las transacciones se conocen mediante el acrónimo ACID , que representa Atomicity (Atomicidad), Consistency (Consistencia), Isolation (Aislamiento) y Durability (Durabilidad). La atomicidad garantiza que todos los efectos de una transacción se apliquen completamente a la base de datos o que no se aplique ninguno; Para lograr esto, los SDBD implementan técnicas como el registro de transacciones (transaction log) y la recuperación mediante deshacer (undo) en caso de fallos. La consistencia asegura que la transacción transforma la base de datos de un estado válido a otro estado válido, respetando todas las restricciones de integridad definidas. El aislamiento garantiza que los efectos de una transacción no sean visibles para otras transacciones concurrentes hasta que la primera se complete, evitando interferencias problemáticas como las lecturas sucias o las actualizaciones perdidas. Finalmente, la durabilidad asegura que una vez que una transacción se confirma (commit), sus efectos persisten permanentemente en la base de datos incluso si el sistema falla posteriormente.

Los SDBD implementan diversos mecanismos para lograr estas propiedades. El control de concurrencia utiliza técnicas como el bloqueo (locking) y el controlmultiversión (MVCC) para asegurar que las transacciones concurrentes no interfieran entre sí. Los bloqueos pueden ser compartidos (para operaciones de lectura) o exclusivos (para operaciones de escritura), y el sistema debe gestionar cuidadosamente los interbloqueos (interbloqueos) que pueden surgir cuando

múltiples transacciones esperan mutuamente por recursos bloqueados por las demás.

El registro de transacciones mantiene un registro histórico de todas las operaciones realizadas, permitiendo recuperar la base de datos a un estado consistente después de un fallo. Las técnicas de registro de escritura anticipada garantizan que los cambios se registren en el registro antes de aplicarse a la base de datos, mientras que los puntos de control periódicos escriben el estado actual de la base de datos en el disco para acelerar la recuperación.

Manejo de Almacenamiento

El gestor de almacenamiento es el componente del SMBD responsable de la interacción con los dispositivos físicos donde se almacenan los datos. Este componente debe traducir las aplicaciones lógicas de almacenamiento en operaciones físicas sobre el disco, gestionando eficientemente el espacio disponible y optimizando el acceso a los datos.

La organización física de los datos varía según el SMBD y las características del sistema operativo, pero generalmente sigue patrones similares. Los datos se almacenan en bloques o páginas de tamaño fijo (típicamente entre 4 y 64 kilobytes), que constituyen la unidad básica de transferencia entre el disco y la memoria. Las tablas se almacenan como colecciones de páginas, y cada página puede contener uno o más registros de la tabla.

Los índices son estructuras de datos adicionales que mejoran significativamente la velocidad de acceso a los registros. Los índices más comunes son los árboles B+ (B-plus tree), que organizan los valores de las columnas indexadas en una estructura jerárquica que permite búsquedas eficientes en tiempo logarítmico. Cuando se crea un índice sobre una columna, el SMBD construye esta estructura adicional que puede utilizarse para localizar rápidamente los registros que cumplen ciertos criterios de búsqueda.

La gestión del espacio de almacenamiento incluye el seguimiento de qué páginas están libres y cuáles están utilizadas, la asignación de nuevas páginas cuando las tablas crecen, y la recuperación de espacio cuando se eliminan registros o tablas. Los SMBD mantienen estructuras internas llamadas mapas de bits (bitmaps) o listas de espacios libres para rastrear esta información.

Las técnicas de optimización del rendimiento del almacenamiento incluyen el almacenamiento en caché (caching) de páginas frecuentemente accedidas en memoria principal, la lectura anticipada (recarga) de páginas que probablemente se necesiten pronto, y la escritura diferida (escritura diferida) que agrupa múltiples

cambios antes de escribirlos al disco. Estas técnicas buscan minimizar el impacto de la lenta velocidad de acceso al disco comparada con la memoria.

Administración de Bases de Datos

La administración de bases de datos es el conjunto de actividades y responsabilidades dirigido a garantizar el funcionamiento correcto, eficiente y seguro de los sistemas de bases de datos en una organización. El profesional responsable de estas tareas se conoce como Administrador de Bases de Datos (DBA, por sus siglas en inglés Database Administrator) y desempeña un papel crítico en cualquier entorno que utilice bases de datos de manera significativa.

Las funciones del DBA se pueden clasificar en varias categorías principales. En primer lugar, la instalación y configuración del software del SDBD, que incluye la selección de los parámetros apropiados para el entorno específico, la configuración de la memoria, los procesos y las conexiones, y la definición de las ubicaciones de los archivos de datos y registro.

En segundo lugar, la administración del esquema de la base de datos, que comprende la creación y modificación de las tablas, índices, restricciones y otros objetos que componen la estructura de la base de datos. Esta función requiere un conocimiento profundo de los requerimientos de negocio y las técnicas de modelado de datos para diseñar estructuras eficientes y escalables.

La gestión de la seguridad es otra responsabilidad fundamental del DBA, que incluye la creación y mantenimiento de cuentas de usuario, la asignación de permisos y roles, la implementación de políticas de contraseñas, y la auditoría de accesos a datos sensibles. Esta función es especialmente crítica en entornos regulados donde el cumplimiento de normativas como GDPR, HIPAA o PCI-DSS es obligatorio.

El monitoreo del rendimiento y la optimización son tareas continuas que el DBA debe realizar para garantizar que el sistema responda adecuadamente a las necesidades de los usuarios. Esto incluye el análisis de consultas lentas, la identificación de cuellos de botella, el ajuste de índices, la reconfiguración de parámetros y otras actividades de optimización.

La gestión de copias de seguridad y recuperación asegura que los datos puedan restaurarse después de cualquier tipo de fallo, ya sea un fallo de hardware, un error humano o un desastre natural. El DBA debe diseñar e implementar estrategias de respaldo apropiadas, realizar pruebas periódicas de recuperación y documentar los procedimientos de emergencia.

La administración de la disponibilidad implica la planificación e implementación de estrategias de alta disponibilidad y recuperación ante desastres, como la replicación, el agrupamiento, las arquitecturas activo-pasivo y las soluciones de respaldo en sitios remotos. Estas capacidades son críticas para las aplicaciones empresariales que requieren disponibilidad continua.

Introducción del Modelo Conceptual de Datos

El modelo conceptual de datos representa una abstracción de alto nivel que permite representar la estructura de información de un sistema de manera independiente de cualquier implementación tecnológica específica. Mientras que el modelo lógico (como el modelo relacional) está orientado a la implementación en un SDBD específico, el modelo conceptual se centra en captar los requerimientos de negocio y la semántica del dominio de aplicación de manera que pueda ser entendida tanto por técnicos como por expertos del negocio.

El modelo conceptual más utilizado es el modelo Entidad-Relación (E/R), desarrollado originalmente por Peter Chen en 1976 y posteriormente extendido y refinado por numerosos autores. Este modelo proporciona un conjunto de conceptos y notaciones que permiten describir el mundo real en términos de entidades, atributos y relaciones, capturando la estructura y semántica de los datos de manera intuitiva y visual.

La importancia del modelo conceptual radica en varios factores fundamentales. Primero, proporciona un medio de comunicación entre los diferentes partes interesadas del proyecto, permitiendo que usuarios de negocio, analistas, diseñadores y programadores utilicen un lenguaje común para discutir los requerimientos de información. Segundo, sirve como punto de partida para el diseño de las estructuras de datos físicos, guiando las decisiones de implementación. Tercero, facilita la identificación de inconsistencias y omisiones en los requerimientos antes de que el sistema sea construido. Cuarto, documenta la estructura semántica de la información de manera que pueda mantenerse y evolucionar con el tiempo.

La construcción de un modelo conceptual generalmente es un proceso iterativo que comienza con la identificación de las entidades principales del dominio, continúa con la identificación de las relaciones entre entidades, y refina progresivamente los detalles mediante la especificación de atributos, restricciones y cardinalidades. Este proceso requiere un conocimiento profundo del dominio de la aplicación y una estrecha colaboración entre los expertos en el negocio y los analistas de datos.

Identificación de Elementos del Modelo E/R

El modelo Entidad-Relación se fundamenta en tres conceptos básicos: entidades, atributos y relaciones . Comprender estos elementos y sus interrelaciones es fundamental para poder construir e interpretar modelos conceptuales de datos.

Una entidad representa un objeto del mundo real que tiene existencia independiente y puede ser identificado de manera única. Las entidades pueden ser objetos físicos (como un cliente, un producto o un empleado) o conceptos abstractos (como un pedido, una factura o una reserva). Cada entidad se caracteriza por tener un conjunto de propiedades y por distinguirse de otras entidades del mismo tipo.

Un conjunto de entidades agrupa a todas las entidades del mismo tipo que existen en el sistema. Por ejemplo, el conjunto de entidades clientes agrupa a todos los clientes de la organización. Es importante distinguir entre la entidad individual (un cliente específico) y el conjunto de entidades (la colección de todos los clientes). En las notaciones típicas, el conjunto de entidades se representan mediante rectángulos con el nombre en singular.

Cada entidad dentro de un conjunto de entidades debe tener características que la distingan de las demás. Estas características distinguidoras se conocen como identificadores o claves . En el modelo E/R, los identificadores se representan generalmente mediante subrayados en los atributos que conforman la clave.

Atributos

Los atributos son las características que describen las entidades y las relaciones. Cada atributo tiene un dominio de valores posibles, denominado dominio del atributo, que especifica el conjunto de valores que el atributo puede tomar. Por ejemplo, el atributo "edad" de una entidad EMPLEADO podría tener como dominio los números enteros entre 18 y 100, mientras que el atributo "nombre" podría tener como dominio el conjunto de todas las cadenas de texto posibles.

Los atributos pueden clasificarse según diferentes criterios. Según su estructura, pueden ser simples (atómicos) o compuestos . Los atributos simples contienen un único valor atómico, como un número o una cadena de texto. Los atributos compuestos están formados por la combinación de varios atributos simples, como una dirección que se compone de calle, número, ciudad, código postal y país. Los atributos compuestos pueden dividirse en sus componentes para obtener valores atómicos.

Según el número de valores que pueden almacenar, pueden ser monovalentes (o simples) o multivalentes . Los atributos monovalentes almacenan un único valor para cada entidad, como el RFC de un contribuyente. Los atributos multivalentes pueden almacenar múltiples valores para una misma entidad, como los teléfonos

de contacto de un cliente. En el modelado conceptual, los atributos multivalentes deben evitarse siempre que sea posible, ya que complican la implementación en el modelo relacional.

Según si su valor puede determinarse a partir de otros atributos, pueden ser derivados o no derivados. Los atributos derivados son aquellos cuyo valor puede calcularse a partir de otros atributos de la misma entidad o de entidades relacionadas. Por ejemplo, la edad de un empleado puede derivarse de su fecha de nacimiento. Los atributos derivados no deben almacenarse en la base de datos, sino calcularse cuando sean necesarios, para evitar redundancias y potenciales inconsistencias.

Relaciones

Una relación representa una asociación entre dos o más entidades. Las relaciones tienen significado semántico en el contexto del dominio de aplicación y generalmente representan acciones o conexiones entre las entidades. Por ejemplo, la relación "compra" entre las entidades CLIENTE y PRODUCTO representa que un cliente adquiere productos.

Al igual que las entidades, las relaciones pueden agruparse en conjuntos de relaciones (relationship sets), que agrupan a todas las instancias de relaciones del mismo tipo. Por ejemplo, el conjunto de relaciones COMPRA agrupa todas las compras realizadas por clientes de productos.

Las relaciones pueden tener atributos descriptivos que proporcionan información adicional sobre la asociación entre las entidades. Por ejemplo, la relación "trabaja_en" entre las entidades EMPLEADO y DEPARTAMENTO podría tener atributos como "fecha_de_inicio" y "cargo" que describen las circunstancias específicas de la asignación del empleado al departamento.

Multiplicidad de Relaciones entre Entidades

La multiplicidad (también llamada cardinalidad) de una relación especifica el número de instancias de una entidad que pueden asociarse con una instancia de otra entidad a través de la relación. Comprender la multiplicidad es fundamental para diseñar estructuras de datos correctas y para implementar las restricciones apropiadas en la base de datos.

Existen tres tipos principales de multiplicidad en las relaciones binarias (entre dos entidades). La relación uno a uno (1:1) indica que cada instancia de una entidad se asocia con exactamente una instancia de la otra entidad. Por ejemplo, la relación entre EMPLEADO y ESCRITORIO en un sistema de asignación de

oficinas podría ser uno a uno si cada empleado tiene su propio escritorio y cada escritorio está asignado a un solo empleado.

La relación uno a muchos (1:N) indica que una instancia de la primera entidad puede asociarse con Múltiples instancias de la segunda entidad, pero cada instancia de la segunda entidad se asocia con exactamente una instancia de la primera. Por ejemplo, la relación entre DEPARTAMENTO y EMPLEADO es sustancialmente uno a muchos, ya que un departamento puede tener muchos empleados, pero cada empleado pertenece a un solo departamento.

La relación muchos a muchos (N:M) indica que Múltiples instancias de cada entidad pueden asociarse con Múltiples instancias de la otra. Por ejemplo, la relación entre ESTUDIANTE y CURSO es muchos a muchos, ya que un estudiante puede inscribirse en muchos cursos y cada curso puede tener muchos estudiantes inscritos.

La multiplicidad de las relaciones tiene implicaciones importantes para el diseño de la base de datos. Las relaciones uno a uno y uno a muchos pueden implementarse incluyendo la clave de la entidad del lado "uno" como atributo en la tabla de la entidad del lado "muchos". Las relaciones muchos a muchos requieren la creación de una tabla intermedia (tabla de intersección) que contenga las claves de ambas entidades.

Relaciones Ternarias y de Mayor Grado

Las relaciones no necesariamente se limitan a dos entidades; pueden asociar tres o más entidades, en cuyo caso se denominan relaciones ternarias (para tres entidades), cuaternarias (para cuatro), o en general n-arias . Por ejemplo, la relación "suministra" entre PROVEEDOR, PRODUCTO y PROYECTO podría representar que un proveedor específico suministra un producto particular a un proyecto cierto, y esta información no podría capturarse adecuadamente mediante dos relaciones binarias separadas.

La interpretación de la multiplicidad en relaciones de mayor grado es más compleja que en las relaciones binarias. Para una relación ternaria, debemos especificar cuántas combinaciones de entidades pueden participar en una instancia de la relación. Por ejemplo, en la relación Suministra entre las entidades Proveedor, Producto y Proyecto, la multiplicidad podría especificar si un proveedor puede suministrar el mismo producto a Múltiples proyectos, o si un proyecto puede recibir el mismo producto de Múltiples proveedores.

En la práctica, las relaciones de grado superior a tres son relativamente raras y suelen simplificarse dividiéndolas en relaciones binarias mediante la introducción de entidades adicionales. Por ejemplo, una relación ternaria entre Estudiante,

Curso y Profesor podría transformarse en una entidad "Inscripción" relacionada con las tres originales.

Roles en las Relaciones

El concepto de rol en las relaciones especifica la función que desempeña cada entidad participante en la relación. Aunque en muchos casos el nombre de la entidad ya indica su función, en relaciones donde una misma entidad participa varias veces con diferentes funciones, es necesario distinguir explícitamente los roles.

Por ejemplo, consideramos la relación "empleo" entre la entidad PERSONA consigo misma. Una persona puede tener un empleador que es otra persona (en el caso de trabajadores independientes que contratan a otros) o una empresa. Para distinguir estos roles, podemos definir dos roles: "empleador" y "empleado", ambos referidos a la entidad PERSONA. Sin esta distinción, el modelo sería ambiguo.

Los roles son especialmente importantes cuando la misma entidad participa múltiples veces en una relación, como en el caso de la relación "matrimonio" entre dos personas, donde definimos los roles "esposo" y "esposa", o en la relación "autorización" entre un empleado y otro empleado donde un empleado puede autorizar a otro para realizar ciertas acciones.

Atributos en las Relaciones

Las relaciones, al igual que las entidades, pueden tener atributos descriptivos que proporcionan información adicional sobre la asociación entre las entidades participantes. Por ejemplo, la relación "trabaja_en" entre las entidades EMPLEADO y PROYECTO podría tener atributos como "fecha_asignacion", "horas_asignadas" y "rol_en_proyecto" que describen las circunstancias específicas de la asignación del empleado al proyecto.

La decisión de incluir atributos en las relaciones frente a crear entidades separadas depende del contexto específico y de consideraciones de diseño. Como regla general, si los atributos describen una asociación entre entidades y no tienen existencia independiente, pueden incluirse en la relación. Sin embargo, si los atributos tienen vida propia o participan en otras relaciones, puede ser preferible crear una entidad separada.

Conversión de Relaciones Multidireccionales a Binarias

Las relaciones que involucran más de dos entidades (relaciones n-arias) pueden ser más difíciles de implementar directamente en el modelo relacional. Por esta

razón, una técnica común de diseño es convertir las relaciones n-arias en un conjunto de relaciones binarias mediante la introducción de una entidad intermedia.

El proceso de conversión consiste en crear una nueva entidad que represente la relación original y establecer relaciones binarias entre esta nueva entidad y cada una de las entidades originales. Los atributos de la relación original se convierten en atributos de la nueva entidad.

Por ejemplo, supongamos que tenemos una relación ternaria "venta" entre las entidades CLIENTE, PRODUCTO y VENDEDOR. Esta relación ternaria podría convertirse en una entidad "TRANSACCION" relacionada con CLIENTE, PRODUCTO y VENDEDOR. Los atributos de la relación original, como la fecha de venta y el precio, se convierten en atributos de la nueva entidad TRANSACCION.

Esta transformación facilita la implementación en el modelo relacional y puede mejorar la comprensión del modelo, aunque también puede aumentar la complejidad del diseño.

Notación del Modelo E/R

El modelo E/R utiliza diversas notaciones visuales para representar los diferentes elementos del modelo. Aunque existen múltiples variantes de notación, las más utilizadas incluyen las propuestas originales de Chen y las variantes utilizadas por herramientas CASE como Oracle Designer, PowerDesigner o ER/Studio.

En la notación de Chen, los conjuntos de entidades se representan mediante rectángulos, los conjuntos de relaciones mediante rombos y los atributos mediante óvalos. Las líneas conectan los rombos con los rectángulos de las entidades participantes. Los atributos que forman parte de la clave primaria se subrayan.

Las cardinalidades se indican escribiendo los números mínimo y máximo en las líneas de conexión. Por ejemplo, la notación (1,N) indica que la entidad puede participar entre una y muchas veces en la relación. Otras notaciones utilizan símbolos diferentes, como barras simples para indicar "uno" y barras triples para indicar "muchos".

Las herencias (en la notación extendida) se representan mediante triángulos o mediante líneas con extremos de flecha que conectan las entidades especializadas con las generalizadas.

Existen también otras notaciones como la notación de "pata de cuervo" utilizada en herramientas como Microsoft Access y PowerDesigner, donde las relaciones se representan con líneas que terminan en símbolos específicos para indicar la

cardinalidad: un extremo simple para "uno", una bifurcación en forma de pata de ave para "muchos".

Extensión del Modelo E/R: Especialización y Generalización

El modelo E/R extendido introduce conceptos adicionales que permiten modelar situaciones más complejas, incluyendo los mecanismos de especialización y generalización para representar jerarquías entre entidades.

La generalización es el proceso de identificar atributos comunes entre diferentes conjuntos de entidades y crear una entidad más general que capture esos atributos compartidos. Por ejemplo, las entidades CLIENTE_INDIVIDUAL y CLIENTE_EMPRESA podrían generalizarse en la entidad CLIENTE, que contendría los atributos comunes como el identificador, la dirección y el teléfono.

La especialización es el proceso inverso: desde una entidad general y crear Subentidades que representan casos específicos con atributos adicionales. Por ejemplo, la entidad EMPLEADO podría especializarse en las subentidades EMPLEADO_TIEMPO_COMPLETO y EMPLEADO_CONTRATO, cada una con atributos específicos adicionales como el salario para los primeros y el período de contratación para los segundos.

En una jerarquía de generalización/especialización, la entidad más general se denomina supertipo (o entidad padre), mientras que las entidades más específicas se denominan subtipos (o entidades hija). Los subtipos heredan automáticamente todos los atributos del supertipo, además de tener sus propios atributos específicos.

La herencia de atributos es el mecanismo por el cual los subtipos reciben automáticamente los atributos definidos en su supertipo. Esta herencia significa que no es necesario redefinir los atributos comunes en cada subtipo, lo que reduce la redundancia y facilita el mantenimiento del modelo. Por ejemplo, si la entidad CLIENTE tiene un atributo "identificador", tanto CLIENTE_INDIVIDUAL como CLIENTE_EMPRESA tendrán este atributo automáticamente.

Existen diferentes tipos de restricciones que pueden aplicarse a las jerarquías de generalización. La totalidad indica si cada ejemplo del supertipo debe pertenecer a algún subtipo completa o puede no pertenecer a ninguno parcial. La disyunción indica si una instancia del supertipo puede pertenecer a más de un subtipo superpuesto de lo sumo a uno dividir. Estas restricciones afectan la forma en que se implementan las jerarquías en la base de datos.

Bases del Modelo Relacional

El modelo relacional es un modelo teórico de datos propuesto por Edgar F. Codd en 1970 que se basa en la teoría de conjuntos y el álgebra relacional. Este modelo proporciona una fundamentación matemática sólida para el almacenamiento y manipulación de datos, y ha sido la base del desarrollo de la mayoría de los sistemas de bases de datos comerciales durante las últimas cuatro décadas.

El elemento central del modelo relacional es el concepto de relación, que puede conceptualizarse como una tabla bidimensional compuesta por filas y columnas. Cada fila representa una tupla o registro, y cada columna representa un atributo. El orden de las filas y columnas no es significativo; lo relevante es que cada tupla sea única y que cada posición en la tabla tenga un valor significativo.

Un esquema de relación se define mediante un nombre y un conjunto de atributos. Por ejemplo, el esquema de la relación EMPLEADO podría ser: EMPLEADO(Nombre, Apellido, RFC, Fecha_Nacimiento, Salario, Departamento). El esquema define la estructura de la relación, mientras que la Instancia de la relación (también llamada extensión) contiene los datos reales.

El dominio de un atributo especifica el conjunto de valores permitidos para ese atributo. Por ejemplo, el dominio del atributo Salario podría ser el conjunto de todos los números reales positivos, mientras que el dominio del atributo Nombre podría ser el conjunto de todas las cadenas de texto de hasta 50 caracteres. Los dominios proporcionan una forma de definir restricciones sobre los valores permitidos y de establecer comparaciones significativas entre atributos.

Una tabla en el contexto del modelo relacional es la implementación física de una relación. Aunque técnicamente relación y tabla son conceptos diferentes (el primero es teórico y el segundo es físico), en la práctica a menudo se utilizan de manera intercambiable.

Una tupla es una fila de la tabla que representa una Instancia específica de la relación. Cada tupla contiene un valor para cada uno de los atributos del esquema. Por ejemplo, una tupla de la relación EMPLEADO podría ser: ("Juan", "Pérez", "PEJJ800101", "1980-01-01", 15000, "Ventas").

Claves en el Modelo Relacional

El concepto de clave es fundamental en el modelo relacional y se utiliza para identificar de manera única cada tupla en una relación. Existen varios tipos de claves que cumplen diferentes funciones.

Una clave es un conjunto de atributos que identifica de manera única cada tupla en una relación. Una relación puede tener múltiples claves candidato, pero cada una debe satisfacer el requisito de unicidad (no existen dos tuplas con los mismos

valores para esos atributos) y el requisito de irreducibilidad (ningún Subconjunto propio de la clave puede identificar las tuplas de manera única).

La clave primaria es la clave candidato seleccionada para identificar las tuplas de la relación. Por convención, se selecciona una clave primaria y el resto de las claves candidato se denominan claves alternas. La clave primaria no puede tener valores nulos y debe tener un valor único para cada tupla.

Una clave foránea (foreign key) es un conjunto de atributos en una relación que referencian la clave primaria de otra relación (o de la misma relación en el caso de autorrelaciones). Las claves foráneas establecen las relaciones entre tablas y son fundamentales para mantener la integridad referencial.

Por ejemplo, en una base de datos con las tablas EMPLEADO y DEPARTAMENTO, la tabla EMPLEADO podría tener un atributo "Departamento_ID" que es una clave foránea que hace referencia a la clave primaria de la tabla DEPARTAMENTO. Esta clave foránea establece la relación entre empleados y departamentos.

Álgebra Relacional

El álgebra relacional es un conjunto de operaciones que permiten manipular datos definidos en el modelo relacional. Estas operaciones toman una o más relaciones como entrada y producen una nueva relación como salida. El álgebra relacional constituye la base teórica para el lenguaje SQL y otros lenguajes de consulta.

La operación de selección (σ , sigma) extrae tuplas que satisfacen una condición específica. Por ejemplo, $\sigma(\text{Salario} > 10000)(\text{EMPLEADO})$ extraería todos los empleados cuyo salario sea mayor a 10000. La selección reduce el número de tuplas pero mantiene todos los atributos.

La operación de proyección (π , pi) extrae atributos específicos de la relación, eliminando los demás. Por ejemplo, $\pi(\text{Nombre}, \text{Apellido})(\text{EMPLEADO})$ extraería únicamente los nombres y apellidos de todos los empleados. La proyección reduce el número de atributos pero mantiene todas las tuplas.

La operación de reunión ($\bowtie$, unir) combina tuplas de dos relaciones basadas en una condición de igualdad en atributos comunes. Por ejemplo, $\text{EMPLEADO} \bowtie \text{DEPARTAMENTO}$ combinaría cada empleado con su departamento correspondiente basándose en el código de departamento. La reunión es una de las operaciones más importantes y frecuentes.

La operación de unión ($\cup$) combina tuplas de dos relaciones con el mismo esquema, eliminando duplicados. La operación de diferencia ($-$) devuelve tuplas

que están en la primera relación pero no en la segunda. La intersección ($\cap$) devuelve tuplas comunes a ambas relaciones.

La operación de producto cartesiano ($\times$) combina cada tupla de la primera relación con todas las tuplas de la segunda. Esta operación produce relaciones con todos los atributos de ambas y es la base para otras operaciones.

Existen otras operaciones adicionales como la división, el renombramiento de atributos, y las funciones de agregación que extienden las capacidades del álgebra relacional básica.

Restricciones del Modelo Relacional

El modelo relacional define varias restricciones de integridad que garantizan la coherencia y corrección de los datos. Estas restricciones son reglas que deben cumplirse en todo momento y son aplicadas por el SDB.

La restricción de la entidad establece que toda relación debe tener una clave primaria y que ningún atributo de la clave primaria puede tener valores nulos. Esta restricción garantiza que cada tupla pueda identificarse de manera única y que no existan tuplas incompletas.

La restricción de integridad referencial establece que los valores de una clave foránea deben corresponderse con valores existentes de la clave primaria referenciada o ser nulos. Esta restricción garantiza la consistencia de las relaciones entre tablas. Por ejemplo, si un empleado tiene un valor en su atributo "Departamento_ID", ese valor debe existir en la tabla DEPARTAMENTO.

La restricción de dominio establece que cada atributo debe contener únicamente valores de su dominio, es decir, del tipo de dato definido para ese atributo. Por ejemplo, un atributo definido como DATE no puede contener texto.

La restricción de integridad semántica (o de negocio) son reglas específicas del dominio de aplicación que no se capturan en las restricciones anteriores. Por ejemplo, "el salario de un empleado no puede ser mayor que el salario de su gerente" o "un pedido debe tener al menos un producto". Estas restricciones considerables se implementan mediante activadores o validación en la aplicación.

Conversión de Diagramas E/R a Modelos Relacionales

La conversión (o mapeo) de un diagrama E/R a un modelo relacional es un proceso sistemático que permite transformar el diseño conceptual en un diseño lógico implementable. Este proceso establece reglas claras para convertir cada elemento del modelo E/R en estructuras del modelo relacional.

Para convertir entidades , cada conjunto de entidades del modelo E/R se transforma en una tabla relacional. Los atributos de la entidad se convierten en columnas de la tabla, y la clave primaria de la entidad se convierte en la clave primaria de la tabla. Por ejemplo, la entidad EMPLEADO con atributos (ID_Empleado, Nombre, Apellido, Salario) se convierte en la tabla: CREATE TABLE EMPLEADO (ID_Empleado PRIMARY KEY, Nombre, Apellido, Salario).

Para convertir relaciones uno a uno , existen dos opciones principales. La primera opción es fusionar las tablas de las entidades relacionadas en una única tabla que contiene todos los atributos de ambas. La segunda opción, más común, es incluir la clave primaria de una de las entidades como clave foránea en la tabla de la otra entidad.

Para convertir relaciones uno a muchos , la tabla del lado "muchos" incluye la clave primaria de la tabla del lado "uno" como clave foránea. Por ejemplo, en la relación entre DEPARTAMENTO (uno) y EMPLEADO (muchos), la tabla EMPLEADO incluiría el atributo ID_Departamento como clave foránea referenciando la tabla DEPARTAMENTO.

Para convertir relaciones muchos a muchos , es necesario crear una tabla de intersección (o tabla asociativa) que contenga las claves primarias de ambas entidades como parte de su clave primaria compuesta. Por ejemplo, la relación ESTUDIANTE-CURSO se convertiría en una tabla INSCRIPCION con las claves ID_Estududiante e ID_Curso como clave primaria compuesta, además de los atributos de la relación.

Para convertir atributos multivalentes, estos se convierten en entidades separadas relacionadas con la entidad original mediante una relación uno a muchos. Por ejemplo, los Múltiples teléfonos de un cliente se convertirían en una tabla TELEFONO relacionada con la tabla CLIENTE.

Para convertir grados de generalización/especialización, existen varias estrategias. Una estrategia es crear una tabla para el supertipo y tablas separadas para cada subtipo que incluyan únicamente sus atributos específicos. Otra estrategia es crear tablas separadas para cada entidad (supertipo y subtipos) con todos los atributos, copiando los atributos comunes. La elección depende de las características específicas del caso.

Normalización

La normalización es el proceso de organizar las tablas de una base de datos relacional para minimizar la redundancia y evitar anomalías de actualización. Este proceso se fundamenta en el concepto de dependencias funcionales y utiliza una

serie de formas normales progresivas para evaluar y mejorar el diseño de la base de datos.

El objetivo principal de la normalización es descomponer las relaciones que contienen información redundante en otras más pequeñas y bien estructuradas, de manera que cada tabla contiene información sobre un único concepto o entidad. Esto permite reducir el espacio de almacenamiento, evitar inconsistencias en los datos y facilitar el mantenimiento de la base de datos.

Anomalías de Inserción, Borrado y Modificación

Antes de comprender la normalización, es importante comprender los problemas que surgen cuando el diseño de la base de datos no está adecuadamente estructurado. Estos problemas se conocen como anomalías de actualización y se manifiestan de tres formas principales.

La anomalía de inserción ocurre cuando es imposible agregar cierta información a la base de datos porque no se conocen otros datos relacionados. Por ejemplo, si en una tabla que combina información de empleados y departamentos quisiéramos agregar un nuevo departamento sin empleados asignados, tendríamos un problema porque la clave primaria requiere un valor de empleado.

La anomalía de borrado ocurre cuando al eliminar información no deseada también se elimina información valiosa que se desea conservar. Por ejemplo, si eliminamos el último empleado de un departamento, también podríamos perder automáticamente la información del departamento porque los datos del departamento estaban almacenados en la misma tabla.

La anomalía de modificación ocurre cuando al actualizar información se produce inconsistencias porque los mismos datos están almacenados en múltiples lugares. Por ejemplo, si la dirección de un departamento está almacenada en la fila de cada empleado asignado a ese departamento, cambiar la dirección requerirá actualizar múltiples filas.

Estas anomalías surgen principalmente debido a la redundancia de datos, donde la misma información se almacena repetidamente en diferentes lugares. La normalización busca eliminar o reducir esta redundancia reorganizando las tablas en estructuras más apropiadas.

Dependencias Funcionales

Una dependencia funcional es una relación entre atributos donde el valor de un atributo (o conjunto de atributos) determina el valor de otro atributo (o conjunto de

atributos). Se denota como $A \rightarrow B$ y se lee como "A determina funcionalmente a B" o "B depende funcionalmente de A".

Formalmente, dice que para cada valor de A existe exactamente un valor de B asociado. Por ejemplo, en una tabla de EMPLEADO, la dependencia funcional $ID_Empleado \rightarrow Nombre, Apellido, Salario$ indica que el identificador del empleado determina funcionalmente el nombre, apellido y salario.

Las dependencias funcionales completas ocurren cuando un atributo no primo depende funcionalmente de toda la clave primaria y no de un subconjunto de ella. Este concepto es fundamental para la segunda forma normal.

Las dependencias transitivas ocurren cuando un atributo depende funcionalmente de otro atributo que no es clave primaria. Por ejemplo, si $ID_Empleado \rightarrow ID_Departamento$ y $ID_Departamento \rightarrow Nombre_Departamento$, entonces $ID_Empleado \rightarrow Nombre_Departamento$ es una dependencia transitiva.

La identificación de todas las dependencias funcionales en una relación es el primer paso del proceso de normalización y requiere un conocimiento profundo de los requerimientos de negocio.

Descomposición de Relaciones

El procesamiento es el proceso de dividir una relación en dos o más relaciones más pequeñas con el objetivo de eliminar anomalías y mejorar la estructura. Una revisión adecuada debe satisfacer ciertas propiedades fundamentales.

La descomposición sin pérdida (lossless-join decomposition) es aquella que permite reconstruir la relación original mediante la reunión natural de las relaciones resultantes. Esto significa que no se debe perder información ni generar información espuria durante el análisis.

La preservación de dependencias es la propiedad que indica que todas las dependencias funcionales originales pueden verificarse en las relaciones resultantes sin necesidad de realizar una unión. Esta propiedad es deseable pero no siempre alcanzable.

Para lograr un buen análisis, es importante seguir las formas normales progresivas, cada una de las cuales establece condiciones específicas que la relación debe cumplir.

Formas Normales

Las formas normales son un conjunto de condiciones progresivas que una relación debe cumplir para considerarse adecuadamente normalizada. Cada forma normal subsana los problemas de la forma normal anterior.

La Primera Forma Normal (1FN) establece que todos los atributos deben contener valores atómicos, es decir, no pueden contener grupos o matrices de valores. Una tabla está en 1FN cuando cada columna contiene un único valor para cada fila y no hay grupos repetitivos.

Por ejemplo, una tabla que tenga una columna "Teléfonos" con múltiples números separados por comas violaría la 1FN porque contiene múltiples valores. La solución es crear una tabla separada para los teléfonos.

La Segunda Forma Normal (2FN) requiere que la tabla esté en 1FN y que todos los atributos no primos dependan funcionalmente de toda la clave primaria, es decir, no existen dependencias parciales. Esto es especialmente relevante para tablas con claves primarias compuestas.

Por ejemplo, si tenemos una tabla Inscripción(ID_Estudiante, ID_Curso, Fecha_Inscripción, Calificación) donde la clave primaria es la combinación de ID_Estudiante y ID_Curso, y existe un atributo "Fecha_Inscripción" que depende solo de ID_Estudiante (cuándo se inscribió el estudiante), entonces tenemos una dependencia parcial que viola la 2FN.

La Tercera Forma Normal (3FN) requiere que la tabla esté en 2FN y que no existan dependencias transitivas, es decir, que ningún atributo no primario depende funcionalmente de otro atributo no primario. Si $A \rightarrow B$ y $B \rightarrow C$ donde C no es clave, entonces C depende transitivamente de A.

Por ejemplo, si en una tabla EMPLEADO tenemos $ID_Empleado \rightarrow ID_Departamento \rightarrow Nombre_Departamento$, entonces Nombre_Departamento depende transitivamente de ID_Empleado, lo que viola la 3FN.

Forma Normal de Boyce-Codd (BCNF)

La Forma Normal de Boyce-Codd (BCNF) es una forma normal más estricta que la 3FN y se utiliza en casos específicos donde las dependencias funcionales son más complejas. Una tabla está en BCNF si, para toda dependencia funcional no trivial $A \rightarrow B$, A es una superclave de la tabla.

La diferencia entre 3FN y BCNF se manifiesta cuando existen múltiples claves candidatas que se superponen o cuando hay dependencias funcionales donde el determinante no es una clave primaria. En estos casos, la 3FN puede no ser suficiente para eliminar todas las anomalías.

Por ejemplo, consideramos una tabla Asignación(Profesor, Curso, Salón) donde las dependencias funcionales son: (Profesor, Curso) $\rightarrow$ Salón y (Salón, Curso) $\rightarrow$ Profesor. Ambas combinaciones son claves candidatas, pero Profesor por sí solo no es una superclave. Esta tabla está en 3FN pero no en BCNF.

Para normalizar a BCNF, puede ser necesario descomponer la relación de manera que no se preserven todas las dependencias funcionales originales, lo que puede ser aceptable en ciertos contextos.

Es importante notar que alcanzar formas normales más altas no siempre es necesario o deseable. En algunos casos, la desnormalización estratégica puede mejorar el rendimiento de consultas frecuentes, especialmente en aplicaciones analíticas. Sin embargo, para sistemas transaccionales, generalmente se recomienda alcanzar al menos la 3FN.

Introducción a SQL

El SQL (Structured Query Language) es el lenguaje estándar para manipular y consultar bases de datos relacionales. Desarrollado originalmente en IBM en la década de 1970 como parte del proyecto System R, SQL se ha convertido en el lenguaje universal para la interacción con bases de datos relacionales y es soportado por prácticamente todos los sistemas comerciales.

SQL es un lenguaje declarativo, lo que significa que especifica qué resultado se desea obtener sin especificar el procedimiento para lograrlo. Esto lo diferencia de los lenguajes procedimentales donde se especifica el algoritmo paso a paso. El SDBD es responsable de determinar la forma más eficiente de ejecutar la consulta.

El lenguaje SQL se divide en varios subconjuntos funcionales. El DDL (lenguaje de definición de datos) incluye comandos para crear, modificar y eliminar objetos de la base de datos. El DML (Lenguaje de manipulación de datos) incluye comandos para insertar, actualizar, eliminar y consultar datos. El DCL (Lenguaje de control de datos) incluye comandos para controlar permisos y seguridad.

Consultas Simples en SQL: Proyección y Selección

Las consultas básicas en SQL utilizan la instrucción SELECT, que en su forma más sencilla permite proyectar atributos específicos de una tabla. La estructura básica es:

- 1- **SELECT** columna1, columna2, ...
- 2- **FROM** tabla;

Por ejemplo, para obtener los nombres y apellidos de todos los empleados:

- 1- **SELECT** Nombre, Apellido
- 2- **FROM** Empleado;

La cláusula DISTINCT elimina duplicados del resultado:

- 1- **SELECT DISTINCT** Departamento
- 2- **FROM** Empleado;

La selección de filas específicas se realiza mediante la cláusula WHERE , que especifica condiciones que deben cumplirse:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Salario > 10000;

Las condiciones en el WHERE pueden combinar múltiples expresiones utilizando los operadores AND , OR y NOT , así como operadores de comparación como = , <> , < , > , <= , >= .

Comparación de Strings

SQL proporciona capacidades específicas para la comparación de cadenas de caracteres mediante el operador LIKE , que permite patrones con comodinas. El símbolo % representa cualquier secuencia de caracteres y el símbolo _ representa cualquier carácter individual.

Por ejemplo, para encontrar todos los empleados cuyo apellido comienza con "García":

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Apellido LIKE 'García%';

Para encontrar empleados cuyo nombre contenga "Juan" en cualquier posición:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Nombre LIKE '%Juan%';

Para encontrar empleados cuyo apellido tenga exactamente 5 letras y comience con 'Pérez' con cuatro letras y un comodín:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Apellido LIKE 'Pérez _';

Algunos sistemas también soportan expresiones regulares para patrones más complejos, aunque esto varía según el SDBD.

Fecha y Hora

Las bases de datos relacionales proporcionan tipos de datos específicos para manejar fechas y horas, así como funciones para manipulación. Las comparaciones directas son posibles cuando se utilizan estos tipos.

Por ejemplo, para encontrar empleados contratados después del 1 de enero de 2020:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Fecha_Contratacion > '2020-01-01';

Para encontrar empleados contratados en un año específico:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** YEAR(Fecha_Contratacion) = 2023;

O usando funciones específicas del sistema:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** EXTRACT(YEAR FROM Fecha_Contratacion) = 2023;

Existen funciones para manipular fechas como DATE_ADD , DATE_SUB , DATEDIFF , entre otras, que permiten realizar cálculos con fechas.

Valor Nulo y Comparaciones

El valor NULL representa la ausencia de valor o un valor desconocido. Las comparaciones con NULL requieren atención especial porque el resultado de cualquier comparación con NULL es desconocido, no verdadero ni falso.

Para verificar si un valor es NULL, debe utilizar IS NULL o IS NOT NULL :

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Departamento IS NULL;
- 4-
- 5- **SELECT ***
- 6- **FROM** Empleado
- 7- **WHERE** Fecha_Baja IS NOT NULL;

Las comparaciones tradicionales como "Columna = NULL" o "Columna <> NULL" nun todos ellos regresarán verdaderas porque NULL representa un valor desconocido.

Para manejar valores NULL en expresiones, existen funciones como COALESCE , que devuelve el primer valor no nulo de una lista, o IFNULL , que devuelve un valor alternativo cuando el valor es NULL:

- 1- **SELECT** IFNULL(Departamento, 'No asignado') **AS** Dept
- 2- **FROM** Empleado;

También existe la función NULLIF , que devuelve NULL cuando dos valores son iguales, y una cantidad diferente en caso contrario.

Valores de Verdad en SQL

En SQL, las condiciones pueden devolver uno de tres valores: TRUE , FALSE y UNKNOWN (desconocido). Este último surge principalmente de comparaciones con valores NULL.

El operador IS se utiliza para comparaciones específicas con valores booleanos en algunos sistemas:

- 1- **SELECT** *
- 2- **FROM** Tabla
- 3- **WHERE** Condicion **IS TRUE**;

En el estándar SQL:1999, se introdujo el tipo de dato BOOLEAN, que puede almacenar valores TRUE, FALSE o NULL.

El operador BETWEEN permite especificar rangos incluidos:

- 1- **SELECT** *
- 2- **FROM** Empleado
- 3- **WHERE** Salario **BETWEEN 10000 AND 20000**;

El operador IN permite verificar pertenencias a un conjunto:

- 1- **SELECT** *
- 2- **FROM** Empleado
- 3- **WHERE** Departamento **IN** ('Ventas', 'Marketing', 'Recursos Humanos');

Orden de Resultados

La cláusula ORDER BY permite ordenar los resultados según uno o más criterios:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **ORDER BY** Apellido **ASC**, Nombre **DESC**;

Por defecto, el orden es ascendente (ASC), pero puede especificarse explícitamente (DESC para orden descendente).

Pueden utilizarse expresiones y posiciones de columnas para el ordenamiento. Por ejemplo, ORDER BY 3 ordena según la tercera columna del resultado.

La cláusula LIMIT (en MySQL, PostgreSQL) o TOP (en SQL Server) limita el número de filas devueltas:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **ORDER BY** Salario **DESC**
- 4- **LIMIT 10**;

Para paginación, puede combinarse con OFFSET:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **LIMIT 10 OFFSET 20**;

Nombres de Tablas y Atributos

Los alias permiten renombrar columnas o tablas en una consulta, lo que es útil para claridad, para evitar ambigüedades en consultas multirelación, y para crear columnas derivadas:

- 1- **SELECT** Nombre **AS** 'Nombre del Empleado',
- 2- Salario * 1.1 **AS** 'Nuevo Salario'
- 3- **FROM** Empleado **AS** E;

Los alias de tabla son especialmente útiles en consultas que involucren múltiples tablas para simplificar la escritura:

- 1- **SELECT** E.Nombre, D.Nombre
- 2- **FROM** Empleado E
- 3- **JOIN** Departamento D **ON** E.ID_Departamento = D.ID;

Consultas que Involucran Más de una Relación

Las consultas multirelación son fundamentales para combinar información de diferentes tablas. Existen varias formas de lograr esto en SQL.

El producto cartesiano combina cada fila de la primera tabla con todas las filas de la segunda:

- 1- **SELECT ***
- 2- **FROM** Empleado, Departamento;

Esto produce una tabla con todos los atributos de ambas, aunque generalmente no es el resultado deseado.

El join es la operación más común para combinar tablas. La sintaxis clásica utiliza la cláusula WHERE para especificar la condición de combinación:

- 1- **SELECT** E.Nombre, D.Nombre
- 2- **FROM** Empleado E, Departamento D
- 3- **WHERE** E.ID_Departamento = D.ID;

La sintaxis ANSI join es más clara y explícita:

- 1- **SELECT** E.Nombre, D.Nombre
- 2- **FROM** Empleado E
- 3- **JOIN** Departamento D **ON** E.ID_Departamento = D.ID;

Existen diferentes tipos de unión. El INNER JOIN devuelve únicamente las filas que tienen correspondencia en ambas tablas. El LEFT JOIN devuelve todas las filas de la tabla izquierda y las filas correspondientes de la derecha (NULL si no hay correspondencia). El RIGHT JOIN es similar pero para la tabla derecha. El FULL OUTER JOIN devuelve todas las filas de ambas tablas.

Interpretación de Consultas Multirelación

Las consultas que involucran múltiples tablas deben interpretarse cuidadosamente para asegurar que el resultados sea el esperado.

Por ejemplo, para encontrar todos los departamentos que tienen al menos un empleado con salario mayor a 15000:

- 1- **SELECT DISTINCT** D.Nombre
- 2- **FROM** Departamento D
- 3- **JOIN** Empleado E **ON** D.ID = E.ID_Departamento
- 4- **WHERE** E.Salario > 15000;

Es importante entender que sin el DISTINCT, si un departamento tiene varios empleados con salario mayor a 15000, el nombre del departamento aparecerá Múltiples veces en el resultado.

Las subconsultas permiten anidar consultas dentro de otras, proporcionando capacidades expresivas adicionales:

- 1- **SELECT ***
- 2- **FROM** Empleado
- 3- **WHERE** Salario > (**SELECT** AVG(Salario) **FROM** Empleado);

Las subconsultas pueden utilizarse en las cláusulas DONDE, DE, EN, EXISTE, entre otras.

Unión, Intersección y Diferencia de Consultas

SQL proporciona operaciones de conjuntos para combinar los resultados de múltiples consultas.

La UNION combina los resultados de dos o más consultas, eliminando duplicados:

- 1- **SELECT** Nombre **FROM** Empleado
- 2- **UNION**
- 3- **SELECT** Nombre **FROM** Cliente;

Para incluir duplicados, se utiliza UNION ALL :

- 1- **SELECT** Nombre **FROM** Empleado
- 2- **UNION ALL**
- 3- **SELECT** Nombre **FROM** Cliente;

La INTERSECT devuelve únicamente las filas que aparecen en ambas consultas:

- 1- **SELECT** Nombre **FROM** Empleado
- 2- **INTERSECT**
- 3- **SELECT** Nombre **FROM** Cliente;

La EXCEPT (o MINUS en algunos sistemas) devuelve las filas de la primera consulta que no aparecen en la segunda:

- 1- **SELECT** Nombre **FROM** Empleado
- 2- **EXCEPT**
- 3- **SELECT** Nombre **FROM** Cliente;

Es importante que las consultas tengan el mismo número de columnas y tipos de datos compatibles para poder aplicar estas operaciones.

Este informe ha cubierto los aspectos fundamentales de las bases de datos, desde los conceptos teóricos básicos hasta las aplicaciones prácticas del lenguaje SQL. A lo largo de las cuatro unidades presentadas, se ha proporcionado una base sólida para comprender y trabajar con sistemas de bases de datos relacionales.

La Unidad 1 desarrolló los fundamentos de los sistemas de bases de datos, visitando conceptos como el modelo de datos, los componentes del SDBD y las responsabilidades del administrador de bases de datos. Estos conceptos son esenciales para comprender cómo funcionan internamente los sistemas de gestión de bases de datos y cómo se organizan para proporcionar capacidades de almacenamiento, recuperación y gestión de datos.

La Unidad 2 introdujo el modelo conceptual Entidad-Relación, que constituye la herramienta fundamental para el diseño de estructuras de datos antes de su implementación física. El dominio de esta notación y sus extensiones, incluyendo la especialización y generalización, permite capturar los requerimientos de negocio de manera precisa y comunicar efectivamente los diseños entre todos los participantes del proyecto.

La Unidad 3 adoptó el modelo relacional, proporcionando la base teórica necesaria para la implementación correcta de bases de datos. La comprensión de conceptos como claves, restricciones de integridad y normalización es indispensable para diseñar bases de datos que minimicen la redundancia y eviten anomalías de actualización.

Finalmente, la Unidad 4 presentó el lenguaje SQL, proporcionando las habilidades prácticas necesarias para interactuar con bases de datos reales. El dominio de las consultas simples, las consultas multirelación y las operaciones de conjunto permite a los profesionales extraer y manipular información de manera efectiva.

En conjunto, estas cuatro unidades proporcionan una formación integral que permite a los profesionales diseñar, implementar y mantener sistemas de bases de datos relacionales de manera competente. El conocimiento de estos fundamentos es cada vez más valioso en un mundo donde la información constituye un activo estratégico para las organizaciones.

Los avances tecnológicos continúan transformando el panorama de las bases de datos, con el surgimiento de bases de datos NoSQL, bases de datos en la nube y arquitecturas distribuidas. Sin embargo, los principios fundamentales

presentados en este informe siguen siendo relevantes y constituyen la base sobre la cual se construyen estas nuevas tecnologías.